

Оптимальные углы QAOA предсказанием машинного обучения

12-кубитная модель Изинга · QAOA глубины $p=5$ · multi-output regression

Задача «Машинное обучение в квантовых вычислениях» (РКЦ, Сколково)

Команда: <имя участника>

1

Задача

- **Дано:** фиксированная матрица J (12x12), 500 векторов h_{train} (и h_{test}) линейных коэффициентов, дифференцируемый симулятор QAOA.py.
- **Предсказать:** для каждого h_{test} — углы ($\gamma_0..4, \beta_0..4$) схемы QAOA глубины 5, максимизирующие $P(\text{ground})$.
- **Метрика:** $P(\text{ground})$ = вероятность основного состояния QAOA-состояния; баллы — по рангу среди воспроизводимых решений:
 $\text{Score} = 70 \cdot (1 - (i-1)/n)$.
- **Ограничения:** Colab «в одну кнопку»; инференс ≤ 10 мин; углы обязаны зависеть от h (константные — 0 баллов).

$$P(\text{ground})_k = \sum_{\{s: E_k(s) = E_k^{\min}\}} |\langle s | \psi_k(\gamma_k, \beta_k) \rangle|^2$$

$$\text{метрика} = (1/500) \sum_k P(\text{ground})_k$$

2

Почему ML: классический цикл — узкое место

- Классический цикл QAOA ищет углы итеративно: тысячи запусков схемы на инстанс — вычислительное узкое место.
- ML-модель $f(h) \rightarrow (\gamma, \beta)$ предсказывает близкие к оптимуму углы сразу: тысячи запусков заменяются одним проходом сети + десятки шагов доводки.
- **Идея решения:** гибрид «ML-тёплый старт + короткая классическая доводка»: сеть задаёт стартовые углы, 40 шагов Adam конкретному h добирают остальное.
- Симулятор QAOA.py дифференцируем — можно обучать сеть сразу через $P(\text{ground})$.

3

Ключевые наблюдения

- **Симметрия J:** точное отражение $i \leftrightarrow 11-i$ — используем симметризованные и антисимметризованные проекции h .
- **Основное состояние:** получается точным перебором $2^{12} = 4096$ конфигураций (микросекунды); минимумы не вырождены.
- **Неуникальность углов:** в ландшафте $P(\text{ground})$ несколько локальных минимумов с сопоставимым качеством — L2-регрессия усредняет по бассейнам и деградирует.
- **Вывод:** обучаем сеть end-to-end с потерей $-P(\text{ground})$ — сеть сама выбирает любой эквивалентный по качеству набор углов.

Пайплайн

1. Метки

поинстансный Adam:
3 рестарта x 250 шагов,
батч = все 500 h



2. Признаки

h + симметрии J +
физика (перебор 2^{12}):
E_min, gap, h.s*, ...



3. Обучение

MLP, end-to-end:
loss = -P(ground)+aux;
450 real + 1000 synth



4. Инференс

f(h) -> углы +
40 шагов Adam
по конкретному h

Валидация: 50 реальных h (holdout из 500) + 500 синтетических $h \sim U[-1,1]^{12}$ — защита от заучивания публичного лидерборда (он считается на h_train).

5

Физические признаки

- **h (12):** вектор линейных коэффициентов.
- **h_sym, h_asym (12+12):** проекции по зеркальной симметрии J — сеть «знает» симметрию задачи.
- **E_min, gap:** энергия основного состояния и зазор до первого возбуждённого (точный перебор 4096 конфигураций).
- **h · s*:** согласованность линейного члена с основным состоянием.
- **||h||, mean|h|, std(h):** характеристика поля.
- Итог: 42 признака на инстанс; все вычисляются за миллисекунды без квантового симулятора.

6

Архитектура модели

- **Ствол:** 3 слоя x 256 нейронов, GELU, dropout 0.05.
- **Угловые головы:** по одной голове на каждый из 5 слоёв QAOA для γ и для β — углы предсказываются как «schedule» по гл (10 выходов), а не как независимые числа.
- **Аух-головы (multi-task):** энергия основного состояния (MSE) и его конфигурация (BCE, 12 бит) — физический сигнал, улучшает обоб
- **Инициализация:** смещения угловых голов — средние значения меток (полная поинстансная оптимизация).

$$\text{loss} = - \text{mean_k } P(\text{ground})(h_k, f(h_k)) + 0.05 \cdot (\text{MSE}(E) + \text{BCE}(s^*))$$

обучение через дифференцируемый QAOA-симулятор из условия (без изменений)

7

Инференс и «полировка»

- Шаг 1: $f(h) \rightarrow (\gamma, \beta)$ — один проход сети по 500 инстансам (мс).
- **Шаг 2 — полировка:** от предсказания сети 40 шагов Adam поинстансно (каждый h дорабатывает свои углы, $\text{loss} = -P(\text{ground})$).
- Тысячи запусков схемы $\rightarrow$ десятки; качество почти полное: сеть + 40 шагов теряют доли процента $P(\text{ground})$ против полной оптимизации.
- **Время:** ~2-3 мин на CPU / секунды на GPU для всех 500 h_{test} (лимит 10 мин). Детерминировано $\rightarrow$ воспроизводимо.
- Углы детерминированно зависят от h — требованию «не константные углы» удовлетворяем конструктивно.

8 Результаты

полная поинстансная оптимизация (потолок)	—	—
сеть (чистая, без полировки), holdout+synth	—	
сеть + полировка, 50 реальных (holdout)	—	
самопроверка на h_train: сеть / сеть+полировка	—	—

Время инференса (500 h, сеть + полировка): — с (лимит 600 с).

Все сиды фиксированы (SEED=42); чекпоинты: data/labels.npz, data/model.pt, data/history.csv.

9

Воспроизводимость и подача

- **Colab «в одну кнопку»:** solution.ipynb самодостаточен — зависимости, J, h_train и QAOA-класс встроены; нужно только приложить и выполнить Run all.
- **Локально:** pip install -r requirements.txt; generate_labels.py -> train.py -> infer.py.
- **Версии:** numpy, torch (в Colab установлен); все сиды SEED=42.
- **Чекпоинты:** метки после каждого рестарта, лучшая модель по валидации, история обучения — повторный запуск продолжает с чекпоинтов.
- **Структура:** README.md (описание и запуск), requirements.txt, solution/ (код), solution.ipynb (подача), presentation.pdf.